



Tecnológico de Monterrey

Comparación de búsqueda de prefijos con Fuerza Bruta y Trie

Franco De Escondrillas Vázquez | A01739410

Octavio Hernández Loyo | A01739304

Ingeniería en Tecnologías Computacionales

Dr. Sandra Eugenia Barajas Montiel

TC2038

31 de agosto, 2026

1. Introducción

En esta actividad se compararon dos estrategias para resolver una búsqueda de prefijos: recorrer todas las palabras por fuerza bruta o almacenarlas en un Trie, una estructura de árbol donde los caracteres de la palabra forman el camino a seguir. El objetivo es implementar y analizar ambas formas de búsqueda por medio de la medición del tiempo así como las ventajas y desventajas de los métodos.

2. Funciones implementadas y complejidades para el Trie

```
C/C++
class TrieNode {
public:
    TrieNode();

    // Transforma patrón a minúsculas sin acentos --O(|p|)
    void transform(string& s);

    // Insertar patrón en el Trie --O(|p|)
    bool insert(const string& s, int idx = 0);

    // Eliminar patrón del Trie --O(|p|)
    bool remove(const string& s, int idx = 0);

    // Seguir un patrón a través del Trie --O(|p|)
    TrieNode* walk(const string& s, int idx = 0);

    // Revisar si un string es parte del Trie --O(|p|)
    bool membershipQuery(const string& s, int idx = 0);

    // Recuperar todos los strings miembros del Trie --O(|n|)
    void produce(vector<string>& results, const string& pathChars = "");

    // Recuperar todos los string miembros de un Subtrie --O(|n|)
    vector<string> complete(const string& prefix);
```

```
private:
    map<char, TrieNode*> children;
    // ASCII de los acentos (segundo byte)
    map<char, char> dictionary {
        {char(161), 'a'}, // á --> a
        {char(169), 'e'}, // é --> e
        {char(173), 'i'}, // í --> i
        {char(177), '~'}, // ñ --> ~
        {char(179), 'o'}, // ó --> o
        {char(186), 'u'}, // ú --> u
        {char(188), 'u'} // ü --> u
    };
    bool is_end = false;
};
```

3. Análisis de resultados entre distintos enfoques

3.1 Tabla de velocidades

La siguiente tabla muestra el promedio de una consulta. Los valores dependen del equipo, compilador y distribución de las palabras; por ello deben interpretarse como una comparación del prototipo probado y no como una medición universal.

Palabras	Fuerza bruta (µs)	Trie (µs)	Más rápido
1,000	527	87	Trie
5,000	692	133	Trie
10,000	3478	7	Trie

Un detalle importante que se debe mencionar es que la complejidad del Trie depende fuertemente del tipo de mapa que se use. En el caso de un **unordered_map**, la complejidad **O(1)** reduce la ejecución drásticamente, comparado con el **O(log(n))** de un **map**.

Otro factor que afecta la ejecución es el tamaño del patrón que se busca. En nuestra implementación de fuerza bruta, la complejidad de buscar un patrón es de **O(n · |p|)**, por lo que buscar patrones de tamaño **|p| = 1** tiende a **O(n)** y compite mejor con la velocidad del Trie. Por otro lado, patrones largos amplifican la diferencia entre ambos algoritmos.

En la siguiente figura, se puede apreciar el test realizado para medir las velocidades de cada algoritmo. Como se observa, el último test, con una palabra de mayor longitud, dio como resultado una diferencia en tiempo de ejecución mayor. Otro punto a considerar es que, como la palabra no se encontró en el Trie, este abortó la búsqueda rápidamente (subtrie de 0), mientras que la fuerza bruta checó todas las palabras.

```
→ A01739304-A01739410_Act2.1 git:(main) × g++ main.cpp Trie.cpp -o x && ./x
Test actual: test_1e3.txt

Buscar con prefijo: 0

Fuerza bruta encontró 12 palabras en 527 µs
Trie encontró 12 palabras en 87 µs
!!!Dado que Trie es un set, al ignorar acentos existen palabras que colapsan (como robo y robó)
→ A01739304-A01739410_Act2.1 git:(main) × g++ main.cpp Trie.cpp -o x && ./x
Test actual: test_5e3.txt

Buscar con prefijo: n

Fuerza bruta encontró 66 palabras en 692 µs
Trie encontró 66 palabras en 133 µs
!!!Dado que Trie es un set, al ignorar acentos existen palabras que colapsan (como robo y robó)
→ A01739304-A01739410_Act2.1 git:(main) × g++ main.cpp Trie.cpp -o x && ./x
Test actual: test_1e4.txt

Buscar con prefijo: longitud

Fuerza bruta encontró 0 palabras en 3478 µs
Trie encontró 0 palabras en 7 µs
!!!Dado que Trie es un set, al ignorar acentos existen palabras que colapsan (como robo y robó)
```

Figura 1. Ejecución en terminal del código con tres testcases distintos.

3.2 Ventajas y Desventajas

3.2.1 Fuerza bruta

Ventajas: La implementación es corta comparada con otros métodos, se necesita poca memoria adicional y el recorrido secuencial es eficiente.

Desventajas: El método revisa todas las palabras en cada consulta y el tiempo crece con el tamaño del conjunto.

3.2.2 Trie

Ventajas: La búsqueda es guiada por el prefijo que se comparten entre varias palabras lo que lo hace muy apropiado para consultas repetidas.

Desventajas: Ocupa una cantidad mayor de memoria adicional, la implementación y uso de nodos es más complejo y la construcción del árbol presenta un mayor costo.

3.3 Costo de memoria de Trie

La fuerza bruta conserva principalmente el vector de cadenas. El Trie necesita además un nodo por cada prefijo distinto, mapas para almacenar hijos, punteros y una bandera de fin de palabra. Aunque compartir prefijos evita duplicar ciertos caracteres, la sobrecarga de cada nodo y del `unordered_map` puede superar ampliamente el tamaño del texto original, especialmente cuando las palabras comparten pocos prefijos.

4. Reflexión final

El Trie es más eficiente que la fuerza bruta cuando se realizan consultas repetidas sobre un conjunto grande de palabras, especialmente cuando los prefijos buscados son largos. En lugar de revisar todas las palabras, el Trie sigue únicamente los caracteres del prefijo, por lo que la búsqueda depende principalmente de la longitud del patrón. Esto permite obtener una ventaja considerable frente a la fuerza bruta, aunque el resultado también depende de la estructura utilizada para almacenar los hijos, como se observó al comparar `map` y `unordered_map`.

El principal costo adicional del Trie es la memoria, ya que necesita almacenar nodos, mapas, punteros y banderas de fin de palabra, aunque algunos prefijos sean compartidos. Este ejercicio se relaciona directamente con aplicaciones reales como buscadores y correctores ortográficos, donde es necesario realizar muchas consultas sobre grandes conjuntos de palabras. Por ejemplo, un buscador puede utilizar los prefijos para ofrecer sugerencias mientras el usuario escribe, mientras que un corrector puede recorrer el Trie para encontrar palabras que coincidan con una entrada parcial o determinar rápidamente si una palabra pertenece al vocabulario.